Lecture 6 : Déployer, profiler et intégrer l'IA embarquée

Moteurs d'inférence, budget latence-énergie, cascade et fusion de décisions, projet EcoTri 4.0

Du modèle dans le navigateur à un tri en temps réel sur le tapis.

Pr. Aissam BERRAHOU

Département Ingénierie des systèmes embarqués et mobiles, Smart Systems Lab
IA en Edge Computing, projet fédérateur EcoTri 4.0, version 2027
ENSIAS, Université Mohammed V de Rabat

aissam.berrahou@ensias.um5.ac.ma

Feuille de route

Déployer sur la cible | Profiler | Économiser l'énergie | Décider sous incertitude | Intégrer la station

- **Déployer sur la cible** : *Que se passe-t-il entre le fichier* `.tflite` *et la première prédiction ?*
- **Profiler** : *Où passent les millisecondes, les octets et les millijoules ?*
- **Économiser l'énergie** : *Comment ne réveiller le gros modèle que quand il le faut ?*
- **Décider sous incertitude** : *Comment combiner vision et audio, et quand refuser de décider ?*
- **Intégrer la station** : *Comment assembler les nœuds en un démonstrateur fiable (jalon J6) ?*

Objectifs

Ce que vous saurez faire en sortant de la salle

À la fin de cette lecture, vous serez capables de :

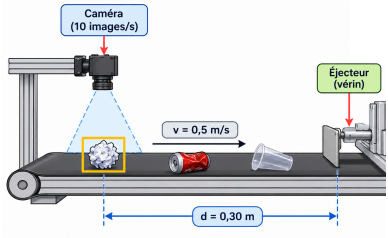
- **Comparer** les moteurs d'inférence embarqués (OpenMV `ml`, TFLite Micro, Edge Impulse EON) et leurs coûts cachés.
- **Reproduire** sur la cible le prétraitement exact de l'entraînement et détecter un *training/serving skew*.
- **Profiler** un pipeline de bout en bout : latence (médiane, p95, pire cas), mémoire, énergie par décision.
- **Concevoir** une cascade de modèles et en calculer la latence et l'énergie moyennes.
- **Fusionner** les décisions vision et audio, **calibrer** les confiances et fixer un seuil de rejet.
- **Intégrer** la station sous forme de machine d'états et publier des alertes JSON.

Le fil rouge et le livrable

- SP6 « Assembler la station » : XIAO → BLE → Portenta → décision → éjecteur et smartphone, en moins de 600 ms.
- **Jalon J6** : démonstrateur complet et **tableau des performances du système** (et non plus du seul modèle).

Le problème déclencheur

180 ms au banc, 900 ms sur le tapis



La situation

- Au jalon J5, le MobileNetV2 α0,35 INT8 affiche **87 %** et **180 ms** sur la Portenta.
- Branché sur la station, un objet sur cinq tombe dans le mauvais bac : l'éjecteur agit **900 ms** après le passage sous la caméra.
- Sur batterie, la XIAO tient **6 h** au lieu des 8 h d'un poste de travail.

À vous (5 min, par équipe)

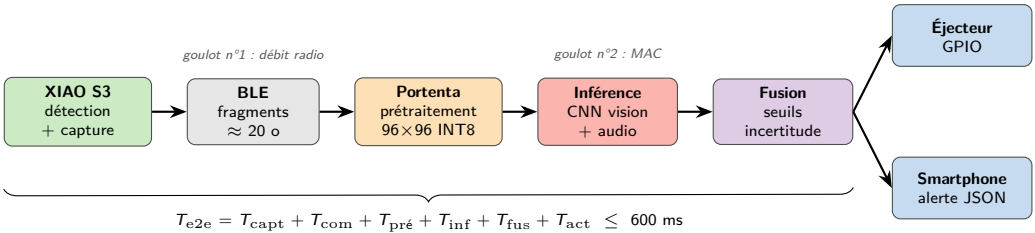
Listez les étapes situées **hors** du modèle qui consomment du temps ou de l'énergie. Laquelle soupçonnez-vous en premier, et comment le vérifieriez-vous ?

Déployer sur la cible

Que se passe-t-il entre le fichier .tflite et la première prédiction ?

Déployer sur la cible

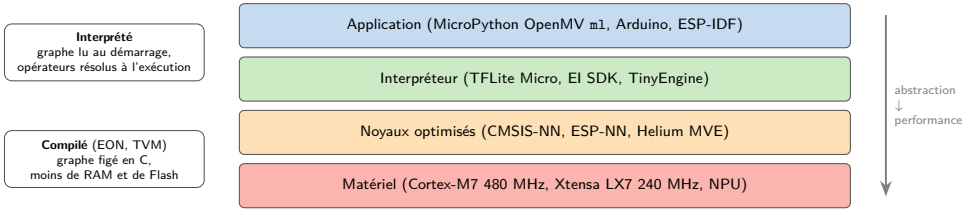
Le système complet : le modèle n'est qu'un maillon



Le jalon J5 mesurait T_{inf} seul. Le jalon J6 mesure T_{e2e} : c'est lui que l'éjecteur subit. La loi d'Amdahl s'applique : diviser T_{inf} par 2 ne sert à rien si la radio pèse 45 % du total.

Déployer sur la cible

La pile logicielle d'un moteur d'inférence embarqué (David et al., Google/Harvard, MLSys 2021)



Interprété (TFLite Micro) : un seul binaire lit n'importe quel `.tflite` ; on paie l'interpréteur (≈ 20–50 Ko de Flash) et les métadonnées en RAM.

Compilé (Edge Impulse EON, TVM, STM32Cube.AI) : le graphe devient du C ; 25 à 55 % de RAM en moins, mais un firmware par modèle.

Déployer sur la cible

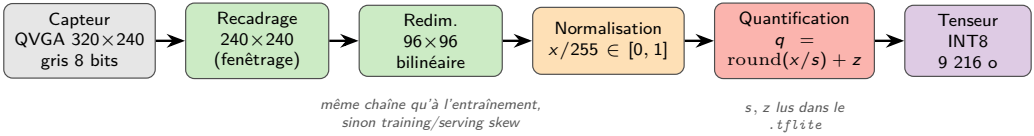
Trois cibles, trois chaînes de déploiement

	Portenta H7 (OpenMV)	**XIAO ESP32-S3 (Arduino)**	**ESP32-CAM (ESP-IDF)**
CPU	Cortex-M7 480 MHz, FPU, DSP	2× Xtensa LX7 240 MHz, SIMD	2× Xtensa LX6 240 MHz
Mémoire	1 Mo SRAM, 8 Mo SDRAM	512 Ko SRAM, 8 Mo PSRAM	520 Ko SRAM, 4 Mo PSRAM
Moteur	TFLM + CMSIS-NN dans le firmware	TFLM + ESP-NN, ou bibliothèque EI	TFLM + ESP-NN
Chargement	fichier sur disque USB / SD	tableau C compilé (`xxd -i`)	tableau C compilé
Itération	secondes (copier, reset)	minutes (compiler, flasher)	minutes

! une même architecture n'a **pas** la même latence sur deux cibles : les noyaux optimisés existent ou non selon l'opérateur. On **mesure** sur chaque cible.

Déployer sur la cible

Le prétraitement : la source n°1 des écarts entre PC et carte



Les pièges classiques

- Recadrage différent (carré central ou image étirée).
- Normalisation $[0, 1]$ contre $[-1, 1]$, ou moyenne/écart-type ImageNet.
- RGB contre BGR, gris contre couleur, ordre HWC.
- Interpolation (plus proche voisin ou bilinéaire).

Training/serving skew (Google, *Rules of ML*, règle 29)

Le modèle ne voit pas la même distribution qu'à l'entraînement : la précision chute sans aucun message d'erreur. Test : envoyer **la même image** au PC et à la carte, comparer les sorties.

Déployer sur la cible

Un squelette d'inférence robuste sur la Portenta


```
import sensor, ml, gc, time
sensor.reset(); sensor.set_pixformat(sensor.GRAYSCALE)
sensor.set_framesize(sensor.QVGA)
sensor.set_windowing((240, 240))      # = recadrage d'entraînement
sensor.set_auto_gain(False)           # exposition figée
sensor.set_auto_exposure(False, exposure_us=8000)
sensor.skip_frames(time=1500)

net = ml.Model("trash_int8.tflite", load_to_fb=True)
print(net.input_shape, net.input_dtype, net.input_scale, net.
      input_zero_point)
for _ in range(5): net.predict([sensor.snapshot()])  # échauffement

def classer(img):
    t0 = time.ticks_us()
    p = net.predict([img])[0].flatten().tolist()
    return p, time.ticks_diff(time.ticks_us(), t0)
```

- **Vérifier** au démarrage forme, type, s et z de l'entrée : une assertion vaut mieux qu'un tri silencieusement faux.
- **Figer** gain et exposition : l'AGC change la luminosité d'une image à l'autre.
- **Échauffer** : les premières inférences remplissent les caches (I-cache, D-cache du M7).

Question flash

?

Votre modèle obtient 87 % dans Colab et 61 % sur la carte avec les mêmes objets. Donnez trois hypothèses classées par probabilité et l'expérience qui tranche chacune.

Question flash

? Votre modèle obtient 87 % dans Colab et 61 % sur la carte avec les mêmes objets. Donnez trois hypothèses classées par probabilité et l'expérience qui tranche chacune.

Réponse

(1) **Prétraitement différent** (normalisation, recadrage) : envoyer une image de test fixe aux deux, comparer les vecteurs de sortie. (2) **Changement de domaine** (éclairage, fond) : évaluer dans Colab les images capturées par la carte. (3) **Mauvais fichier** (float au lieu d'INT8, `labels.txt` dans un autre ordre) : imprimer `net` et l'ordre des classes.

Profiler

Où passent les millisecondes, les octets et les millijoules ?

Profiler

Mesurer, pas deviner : la méthode MLPerf Tiny (Banbury et al., Harvard, 2021)

Les quatre grandeurs du jalon J6

- **Latence** : médiane, p95 et **pire cas** sur $N \geq 100$ essais ; c'est le pire cas qui dimensionne l'éjecteur.
- **Débit** : objets traités par seconde (peut dépasser $1/T_{\text{e2e}}$ si l'on recouvre les étapes).
- **Mémoire** : Flash du modèle, arène (pic), tas libre minimal.
- **Énergie par décision** : $E = \int P\,dt$ sur un cycle complet.

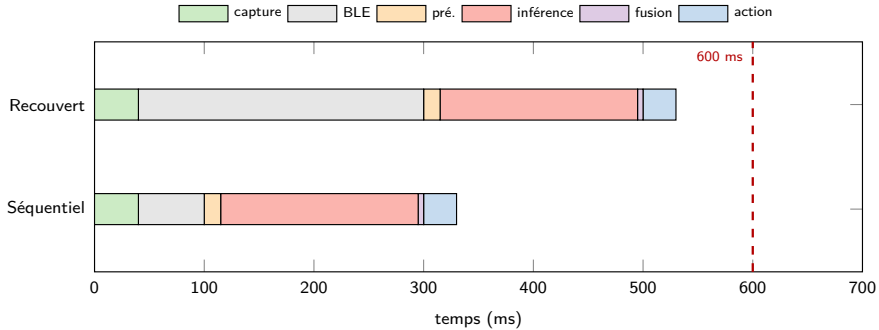
MLPerf Tiny en bref

Benchmark industriel et académique (Harvard, MLCommons) : 4 tâches de référence, mesure de la latence **et** de l'énergie avec un protocole fixé (moniteur d'énergie, déclenchement par GPIO).

> **!** un horodatage par étape ($t_0 \ldots t_5$), une broche GPIO basculée à chaque étape pour l'oscilloscope.

Profiler

Le diagramme de Gantt du pipeline : où agir ?



En séquentiel, la **radio** (envoi d'une image fragmentée) domine. Solution : la XIAO envoie une imagette 96×96 (9 Ko) ou ses caractéristiques, et l'envoi de l'objet $n+1$ recouvre l'inférence de l'objet n.

! si une étape représente une fraction f du temps et qu'on l'accélère d'un facteur k, le gain global est $1/((1-f)+f/k)$: on attaque d'abord la plus grosse barre.

Profiler

Un profileur minimal en MicroPython


```
import time
from pyb import Pin
sonde = Pin("D5", Pin.OUT_PP)          # vers l'oscilloscope

class Profil:
    def __init__(self): self.t = {}
    def top(self, nom):
        sonde.value(not sonde.value())
        self.t.setdefault(nom, []).append(time.ticks_us())
    def rapport(self, etapes):
        for a, b in zip(etapes, etapes[1:]):
            d = sorted(time.ticks_diff(y, x) / 1000
                       for x, y in zip(self.t[a], self.t[b]))
            n = len(d)
            print("%-10s med %6.1f  p95 %6.1f  max %6.1f ms"
                  % (b, d[n//2], d[int(.95*n)], d[-1]))
```

- `ticks_us` : résolution 1 µs, pas d'allocation dans la boucle mesurée.
- `gc.collect()` **avant** la boucle : un ramasse-miettes pendant la mesure crée des pics de plusieurs ms.
- Couper le **Frame Buffer** de l'IDE : l'envoi USB de l'image fausse la latence.

$$T_{\text{inf}} \approx \frac{N_{\text{MAC}}}{\eta\, f_{\text{CPU}}} \qquad E_{\text{inf}} \approx P_{\text{actif}}\, T_{\text{inf}}$$

- η : MAC par cycle effectifs. Cortex-M7 + CMSIS-NN INT8 : $\eta \approx 0{,}15$–$0{,}5$; float32 sans noyaux : $\eta < 0{,}05$.
- Exemple : 11 M MAC à 480 MHz et $\eta = 0{,}13$ $\Rightarrow \approx 176$ ms.
- Un η mesuré très inférieur signale un opérateur non optimisé (repli sur le noyau de référence).

Modèle *roofline* (Williams, Waterman, Patterson, UC Berkeley, 2009)

Une couche est limitée soit par le **calcul**, soit par la **bande passante mémoire**. Sur MCU, les couches depthwise et la SDRAM externe sont souvent limitées par la mémoire : placer l'arène en SRAM interne (DTCM) peut valoir plus qu'un meilleur noyau.

Question flash

?

Mesures sur la station : capture 40 ms, BLE 260 ms, prétraitement 15 ms, inférence 180 ms, fusion 5 ms, action 30 ms. On peut soit diviser l'inférence par 2 (modèle α0,25), soit diviser le temps BLE par 4 (imagette). Quel est le gain de chaque option ?

Question flash

?

Mesures sur la station : capture 40 ms, BLE 260 ms, prétraitement 15 ms, inférence 180 ms, fusion 5 ms, action 30 ms. On peut soit diviser l'inférence par 2 (modèle α0,25), soit diviser le temps BLE par 4 (imagette). Quel est le gain de chaque option ?

Réponse

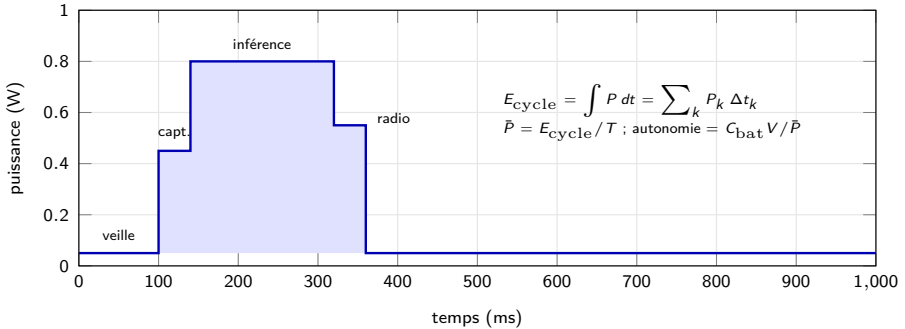
Total actuel : 530 ms. Inférence $/2$: $530 - 90 = 440$ ms (gain 17 %, et perte de précision probable). BLE $/4$: $530 - 195 = 335$ ms (gain **37 %**, sans toucher au modèle). On commence par la radio : « envoyer la décision, pas l'image » (lecture 3).

Économiser l'énergie

Comment ne réveiller le gros modèle que quand il le faut ?

Économiser l'énergie

Profil de puissance d'un cycle de décision



Exemple : $0{,}45 \times 40 + 0{,}8 \times 180 + 0{,}55 \times 40$ ms $\approx$ **184 mJ** actifs, plus $0{,}05 \times 640$ ms $= 32$ mJ de veille. Avec un objet par seconde, $\bar{P} \approx 0{,}22$ W.

! lire un mot en DRAM coûte $\approx 100\times$ plus qu'une addition 32 bits, et émettre un octet par radio encore davantage : moins on déplace de données, moins on consomme.

Économiser l'énergie

Les leviers de la basse consommation

Faire moins

- Déclencher sur événement (capteur IR, différence d'images) plutôt qu'en continu.
- Abaisser la résolution et la cadence quand le tapis est vide.
- Envoyer une décision de 20 octets au lieu d'une image de 9 Ko.

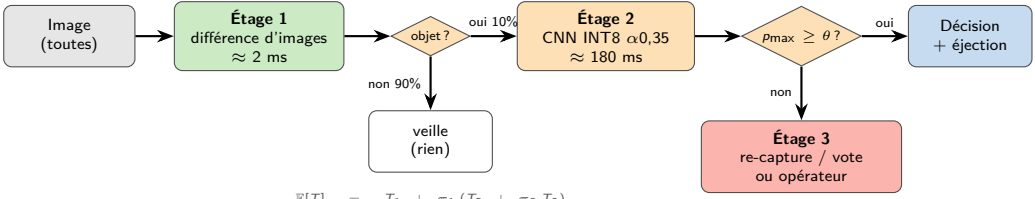
Le faire plus efficacement

- **Race-to-sleep** : finir vite à pleine fréquence puis dormir.
- **DVFS** : baisser f et V ($P_{\text{dyn}} \propto CV^2 f$) quand l'échéance est lâche.
- Modes de veille : `machine.lightsleep()`, `deepsleep()` ; réveil par GPIO ou RTC.

! si la puissance statique (fuites, SDRAM, capteur) est forte, finir vite et tout couper gagne ; si elle est faible, ralentir à tension réduite gagne. Seule la mesure tranche.

Économiser l'énergie

La cascade de modèles (Viola & Jones, 2001 ; BranchyNet, Harvard, 2016)



$$\mathbb{E}[T] = T_1 + \pi_1 (T_2 + \pi_2 T_3)$$

avec π_i la probabilité de passer à l'étage suivant

Un étage bon marché filtre la majorité des cas « faciles » ; le coûteux n'est appelé que pour les autres. Sortie anticipée (*early exit*) : même idée **à l'intérieur** d'un réseau.

Exemple chiffré

$\pi_1 = 0{,}1$, $T_1 = 2$ ms, $T_2 = 180$ ms, $\pi_2 = 0{,}15$, $T_3 = 400$ ms :
$\mathbb{E}[T] = 2 + 0{,}1(180 + 0{,}15 \times 400) = \mathbf{26}$ **ms**, contre 180 ms sans cascade.

Économiser l'énergie

Étage 1 : la détection de présence en quelques millisecondes

```python
import sensor, image
sensor.set_framesize(sensor.QQVGA)        # 160x120 : 19 Ko
fond = sensor.snapshot().copy()           # tapis vide
def objet_present(seuil=12, frac=0.04):
    img = sensor.snapshot()
    img.difference(fond)                  # |img - fond|
    st = img.get_statistics()
    return st.mean() > seuil * frac * 25, img
# en boucle : étage 2 seulement si objet_present()
```

- Coût $\approx$ un passage sur 19 200 pixels : 1 à 3 ms.
- Mettre à jour le fond lentement (α-blending) pour suivre l'éclairage.
- Sur la XIAO : même logique, ou capteur IR sur interruption, et `deepsleep` entre deux objets.

un étage 1 trop sensible réveille l'étage 2 pour rien (énergie) ; trop strict, il rate des objets (rappel). On le règle sur la courbe ROC, en privilégiant le **rappel**.

Question flash

?

La XIAO consomme 300 mW active et 0,5 mW en deep sleep. Un cycle actif dure 120 ms et l'on traite un objet toutes les 2 s. Batterie 1 000 mAh à 3,7 V. Autonomie ? Et sans veille (active en permanence) ?

Question flash

? La XIAO consomme 300 mW active et 0,5 mW en deep sleep. Un cycle actif dure 120 ms et l'on traite un objet toutes les 2 s. Batterie 1 000 mAh à 3,7 V. Autonomie ? Et sans veille (active en permanence) ?

Réponse

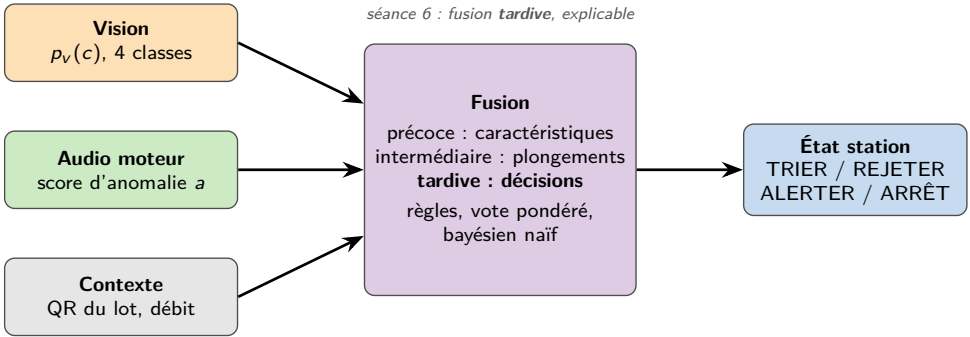
$\bar{P} = (300 \times 0{,}12 + 0{,}5 \times 1{,}88)/2 \approx 18{,}5$ mW. Énergie : 3,7 Wh. Autonomie $\approx 3{,}7/0{,}0185 \approx$ **200 h**. Sans veille : $3{,}7/0{,}3 \approx$ **12 h**. Le **rapport cyclique** compte plus que l'optimisation du modèle.

Décider sous incertitude

Comment combiner vision et audio, et quand refuser de décider ?

Décider sous incertitude

Fusion de décisions : trois niveaux (Kittler et al., IEEE TPAMI 1998)



Précoce : on concatène les caractéristiques, un seul modèle. **Tardive** : chaque modèle décide, on combine les sorties (somme, produit, vote, règles).

Pour EcoTri, la fusion **tardive** est retenue : modèles entraînés séparément (lectures 4 et 5), capteurs asynchrones, décision explicable à l'opérateur.

Pour M classifieurs donnant $p_m(c \mid x_m)$, avec des capteurs supposés indépendants :

$$\text{produit} : \hat{c} = \arg\max_c \; p(c)^{1-M} \prod_m p_m(c \mid x_m)$$

$$\text{somme} : \hat{c} = \arg\max_c \sum_m w_m \, p_m(c \mid x_m)$$

- Produit : optimal si indépendance et probabilités **bien calibrées**.
- Somme pondérée : plus robuste aux erreurs d'un capteur (Kittler, 1998).
- Vote majoritaire : n'utilise que les décisions dures.

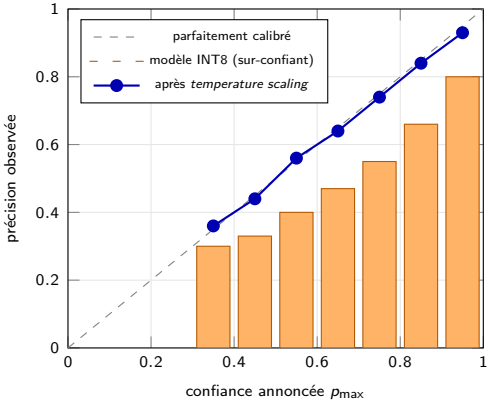
Dans EcoTri : fusion hétérogène

La vision dit **quoi** (matériau), l'audio dit **dans quel état** est la machine. On les combine par **règles prioritaires** :

1. anomalie moteur $a > \tau \Rightarrow$ ARRÊT ;
2. sinon $p_{\max} \geq \theta \Rightarrow$ TRIER ;
3. sinon DOUTE (re-capture, vote sur 2 vues).

Décider sous incertitude

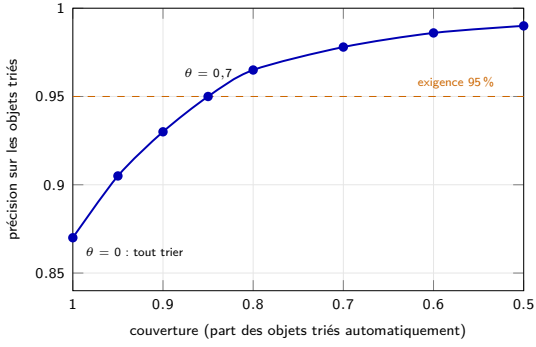
Un réseau est souvent sur-confiant : la calibration (Guo et al., Cornell, ICML 2017)



- Diagramme de fiabilité : on regroupe les prédictions par confiance et on compare à la précision réelle.
- **ECE** $= \sum_b \frac{|B_b|}{n} |\mathrm{acc}(B_b) - \mathrm{conf}(B_b)|$.
- **Temperature scaling** : $p = \mathrm{softmax}(z/T)$, un seul paramètre T appris sur la validation ; ne change pas l'argmax.
- Sur la Portenta, la sortie est déjà un softmax INT8 : on recalcule les logits ou on applique T avant le softmax à la conversion.

Décider sous incertitude

Savoir s'abstenir : classification sélective (Geifman & El-Yaniv, Technion, NeurIPS 2017)



- On ne trie automatiquement que si $p_{\max} \geq \theta$; sinon l'objet va au bac « inconnu » ou à l'opérateur.
- Monter θ : la **précision** sur les objets triés monte, la **couverture** baisse.
- Autres scores d'incertitude : marge $p_{(1)} - p_{(2)}$, entropie $-\sum p \log p$.

> **!** par le **coût** : un métal envoyé au plastique coûte plus cher qu'un passage manuel. On minimise $C = c_{\text{err}}\, N_{\text{err}} + c_{\text{rej}}\, N_{\text{rej}}$ sur la validation.

Décider sous incertitude

Fusion temporelle : plusieurs vues valent mieux qu'une

```python
import math
def fusion_log(vues):              # vues : liste de vecteurs p
    s = [0.0] * len(vues[0])
    for p in vues:
        for c, pc in enumerate(p):
            s[c] += math.log(max(pc, 1e-4))
    m = max(s); e = [math.exp(v - m) for v in s]
    z = sum(e); return [v / z for v in e]

def decider(p, a, theta=0.7, tau=0.8):
    if a > tau:                return "ARRET", None
    k = p.index(max(p))
    if p[k] >= theta:          return "TRIER", k
    return "DOUTE", k
```

- Deux vues du même objet (avant, pendant le passage) : somme des log-probabilités = règle du produit.
- Hypothèse d'indépendance optimiste (même objet, même éclairage) : on reste prudent sur θ.
- Un **plancher** (10^{-4}) évite qu'une vue à 0 annule tout.

Question flash

?

Deux vues d'un objet donnent (carton, métal, plastique, verre) : $p^{(1)} = (0{,}10\,;0{,}55\,;0{,}30\,;0{,}05)$ et $p^{(2)} = (0{,}05\,;0{,}60\,;0{,}30\,;0{,}05)$. Avec $\theta = 0{,}7$, que décide-t-on sur une vue ? Sur la fusion produit ?

Question flash

? Deux vues d'un objet donnent (carton, métal, plastique, verre) : $p^{(1)} = (0{,}10\,;0{,}55\,;0{,}30\,;0{,}05)$ et $p^{(2)} = (0{,}05\,;0{,}60\,;0{,}30\,;0{,}05)$. Avec $\theta = 0{,}7$, que décide-t-on sur une vue ? Sur la fusion produit ?

Réponse

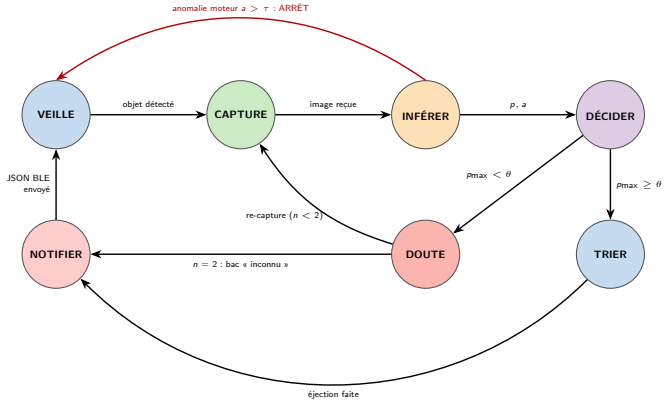
Une vue : $p_{\max} = 0{,}55$ ou $0{,}60 < 0{,}7$: DOUTE. Produit : $(0{,}005\,;0{,}33\,;0{,}09\,;0{,}0025)$, normalisé $\approx (0{,}012\,;0{,}77\,;0{,}21\,;0{,}006)$: $0{,}77 \geq 0{,}7$, **TRIER métal**. La fusion a levé le doute, à condition que les vues soient réellement informatives et le modèle calibré.

Intégrer la station

Comment assembler les nœuds en un démonstrateur fiable (jalon J6) ?

Intégrer la station

La station comme machine d'états



Chaque état a une **échéance** et un **chien de garde** : si l'image n'arrive pas en 400 ms, on revient en VEILLE et on journalise l'erreur. Pas de `while True` monolithique : une fonction par état, une table de transitions testable.

Intégrer la station

Le message d'alerte : un contrat entre nœuds

```
{"v":1, "id":1287, "t":5423110,
 "lot":"L2026-11-A",
 "etat":"TRIER", "classe":"metal",
 "p":0.83, "vues":2,
 "audio":{"a":0.12, "etat":"normal"},
 "lat":{"e2e":312, "inf":178},
 "mdl":"mnv2a035-int8-v3"}
```

197 octets : MTU BLE négociée de 247 ou deux notifications.

- **Version** du schéma (`v`) et du modèle (`mdl`) : indispensables pour la traçabilité et la lecture 7 (MLOps).
- **Identifiant** croissant : le smartphone détecte les pertes.
- Latences incluses : la station **s'auto-mesure** en production.
- Format compact possible (CBOR, binaire) si la bande passante manque.

Intégrer la station

Fiabilité : ce qui casse un démonstrateur

Défaillances typiques

- Fragment BLE perdu : image incomplète, inférence sur des zéros.
- Fragmentation du tas MicroPython après des heures : `MemoryError`.
- Déconnexion du smartphone pendant une notification.
- Dérive de l'éclairage au cours de la journée.

Parades

- Somme de contrôle (CRC-16) et compteur de fragments ; rejet si incomplet.
- Tampons pré-alloués, `gc.collect()` entre deux objets, `WDT` matériel.
- File d'attente des alertes, ré-émission à la reconnexion.
- Recalibrage du fond et contrôle de la luminosité moyenne.

! le jalon J6 exige **30 min** de fonctionnement continu sans redémarrage : c'est là qu'apparaissent les fuites mémoire et les pertes radio.

Intégrer la station

Le tableau des performances du système (jalon J6)

Indicateur	Exigence (J1)	Mesure	Méthode
Latence bout en bout (p95 / max)	$\leq$ 600 ms	...	horodatages + GPIO
Débit soutenu	$\geq$ 1 objet/s	...	30 min, compteur
Précision triée / couverture	$\geq$ 95 % / $\geq$ 85 %	...	40 objets étiquetés
Rappel métal	$\geq$ 90 %	...	matrice de confusion
Énergie par décision (XIAO)	à minimiser	...	USB-mètre / INA219
Pertes BLE	$<$ 1 %	...	identifiants manquants
Tas libre minimal	$>$ 50 Ko	...	`gc.mem_free()`

! on ne note plus « le modèle fait 87 % » mais « la station trie 92 % des objets à 96 % de précision, en 340 ms au p95, pendant 30 min ».

Question flash

?

Votre démonstrateur : 340 ms au p95, mais 1 fois sur 50 un pic à 1,4 s, toujours après plusieurs minutes. Les objets concernés tombent dans le mauvais bac. Diagnostic probable, preuve expérimentale, et deux corrections (l'une logicielle, l'une système) ?

Question flash

?

Votre démonstrateur : 340 ms au p95, mais 1 fois sur 50 un pic à 1,4 s, toujours après plusieurs minutes. Les objets concernés tombent dans le mauvais bac. Diagnostic probable, preuve expérimentale, et deux corrections (l'une logicielle, l'une système) ?

Réponse

Probable : **ramasse-miettes** MicroPython ou reconnexion BLE. Preuve : journaliser `gc.mem_free()` et l'état BLE avec chaque latence, puis corréler. Logiciel : pré-allouer les tampons et appeler `gc.collect()` en VEILLE. Système : l'éjecteur agit sur **horodatage** (position calculée de l'objet) et, si l'échéance est dépassée, l'objet part au bac « inconnu » au lieu d'un bac faux.

À retenir

- **Le système, pas le modèle** : on mesure T_{e2e} au pire cas ; la loi d'Amdahl dit où agir (souvent la radio, rarement le modèle).
- **Déployer**, c'est aussi reproduire **exactement** le prétraitement et vérifier forme, type, s, z au démarrage.
- **Profiler** : médiane, p95, max sur ≥ 100 essais ; mémoire et énergie par décision ; estimer d'abord avec $N_{\mathrm{MAC}}/(\eta f)$.
- **Énergie** : le rapport cyclique domine ; cascade et déclenchement par événement divisent le coût moyen.
- **Décider** : fusion tardive par règles, confiances **calibrées**, seuil de rejet choisi par le coût des erreurs.
- **Intégrer** : machine d'états, chiens de garde, message JSON versionné, test d'endurance.

Glossaire et références

Terme	Définition courte
T_{e2e}	latence de bout en bout
p95	95e centile des latences
Cascade	modèles en série, du moins au plus coûteux
ECE	erreur de calibration attendue
Couverture	part des cas décidés automatiquement
DVFS	ajustement tension-fréquence
WDT	chien de garde matériel

- R. David et al., « TensorFlow Lite Micro : Embedded Machine Learning for TinyML Systems », MLSys 2021 (Google, Harvard).
- C. Banbury et al., « MLPerf Tiny Benchmark », NeurIPS Datasets & Benchmarks 2021 (Harvard, MLCommons).
- C. Guo, G. Pleiss, Y. Sun, K. Weinberger, « On Calibration of Modern Neural Networks », ICML 2017 (Cornell).
- Y. Geifman, R. El-Yaniv, « Selective Classification for Deep Neural Networks », NeurIPS 2017 (Technion).
- J. Kittler, M. Hatef, R. Duin, J. Matas, « On Combining Classifiers », IEEE TPAMI, 1998.
- P. Viola, M. Jones, « Rapid Object Detection using a Boosted Cascade », CVPR 2001 ; S. Teerapittayanon, B. McDanel, H. T. Kung, « BranchyNet », ICPR 2016 (Harvard).
- S. Williams, A. Waterman, D. Patterson, « Roofline », CACM 2009 (UC Berkeley) ; M. Horowitz, ISSCC 2014 (Stanford).
- V. Janapa Reddi, *Machine Learning Systems*, chap. « On-Device Learning », « Benchmarking AI », « ML Operations », mlsysbook.ai (Harvard).
- Docs : OpenMV (`ml`, `machine`), Edge Impulse (EON Compiler), Espressif ESP-NN, Arduino Portenta H7.

Prochaine étape

Tutorial 6 : pipeline instrumenté, cascade et fusion sur la Portenta. **TP 6** : station complète XIAO → BLE → Portenta → smartphone et jalon J6.